

# JABANGSTORE — MASTER CLAUDE BUILD PROMPT

*Complete product specification and phased implementation instructions*

## IMPORTANT: How Claude must use this document

This document is the permanent source of truth for building JabangStore. Do not try to implement the entire application in one response. Work in phases. Before starting any phase, inspect the existing project, database migrations, routes, components, and TODO/status documentation. Preserve completed work. Do not rewrite working features without a clear reason.

- Build JabangStore from scratch; assume there is no existing application unless files in the current project prove otherwise.
- Use the specification below as the product contract.
- Implement one phase at a time and verify it before moving forward.
- If context or token limits are reached, stop safely, summarize exactly what was completed, list remaining work, and leave the project in a runnable state.
- Never replace working code with mockups or placeholder screens when the feature is supposed to be functional.
- Never expose secrets, service-role keys, passwords, or private credentials in frontend code.
- Use environment variables for secrets.
- Do not ask unnecessary questions when the requirements are already specified. Make sensible engineering decisions and document them.
- After each phase, update a project status file such as TODO.md/BUILD\_STATUS.md so another Claude session can continue without re-explaining the project.

## 1. Product identity

- Product name: JabangStore
- Replace every previous occurrence of JabangHub/Jabang Hub with JabangStore in the application, UI copy, metadata, documentation, filenames where appropriate, and branding.
- Do not rename unrelated technical identifiers if doing so would create unnecessary breakage; where a database/table/code identifier already exists, use a safe migration only when needed.
- Target market: small and medium businesses, especially retail shops and local merchants.
- Primary currency: GMD (Gambian Dalasi).
- Primary language: English.
- Brand colors: Deep Green #0B3D2E, Gold #D4AF37, White.
- Primary font: Poppins.
- Design direction: modern, professional, trustworthy, clean, fast, simple, business-focused, and easy for non-technical shop owners.

## 2. Core architecture

- Frontend: React + TypeScript + Vite.

- Styling: Tailwind CSS with a reusable design system.
- Backend/database/auth/storage: Supabase.
- Database: PostgreSQL through Supabase.
- Authentication: Supabase Auth.
- Authorization/security: Supabase Row Level Security (RLS).
- File/image storage: Supabase Storage.
- Application must be a multi-tenant SaaS: many businesses use the same application while each business can access only its own data.
- Design the schema so business\_id/tenant isolation is enforced at the database/security layer, not merely by hiding records in the UI.
- Use reusable components and services; avoid duplicated business logic.
- Use proper loading, empty, error, success, confirmation, and validation states.

### 3. Multi-business SaaS model

JabangStore is one platform serving multiple businesses. Each business must have isolated data. A business may have an owner/admin and multiple staff users with different permissions.

- Business/company profile.
- Business status: pending, active, suspended, rejected, or other clearly defined states as needed.
- Subscription/plan fields should be designed so plans can be added later without rebuilding the application.
- Business-specific products, categories, customers, suppliers, purchases, sales, inventory, reports, settings, and audit records.
- All tenant-sensitive queries must be protected by RLS.
- A user must never be able to change a business\_id in the browser to access another business's data.
- Support future multi-branch operation: business → branches/locations → users and transactions where practical.
- Keep the architecture ready for scaling without overengineering the first version.

### 4. User roles and permissions

- Super Admin / platform owner: manage the JabangStore platform, businesses, approvals, account status, plans, platform-level information, and platform audit information.
- Business Owner/Admin: manage their business, users, products, inventory, customers, suppliers, settings, reports, and sales according to permissions.
- Manager: operational management with configurable permissions.
- Cashier: POS sales and permitted customer/product actions.
- Inventory/Stock staff: inventory and stock operations.
- Additional roles may be added through a permissions system rather than hard-coding every screen.

### 5. Database requirements

Create a production-minded Supabase PostgreSQL schema with migrations. At minimum, design tables/relations for:

- profiles / user profiles
- businesses

- business\_members or user-business memberships
- roles and permissions where appropriate
- branches/locations
- product categories
- products
- product images
- inventory/stock levels
- inventory movements/adjustments
- customers
- suppliers
- sales
- sale items
- payments
- purchases
- purchase items
- expenses where needed for reporting
- returns/refunds
- audit logs
- business settings
- subscription/plan/account-status fields prepared for future use
- Use UUID primary keys where appropriate.
- Use timestamps consistently.
- Use foreign keys and appropriate indexes.
- Use numeric/decimal types for money; never use floating-point arithmetic for financial amounts.
- Store monetary values in GMD with a consistent precision policy.
- Use database constraints to protect data integrity.
- Create indexes for frequent filters such as business\_id, branch\_id, product search fields, sale dates, customer IDs, and status.
- Create safe migrations and seed data for development/demo use.
- Create and test RLS policies for every tenant-sensitive table.

## 6. Authentication

- Implement secure Supabase email/password authentication.
- Provide login, logout, session persistence, protected routes, loading state, and unauthorized handling.
- Do not hard-code any real owner password in source code.
- Use environment variables for Supabase URL and anon/public key.
- Create secure role/business membership checks.
- Provide appropriate account/profile information after login.
- Super Admin access must be separately protected by role/authorization.

## 7. Business registration and approval workflow

- Public/business registration request form.

- Collect appropriate business information such as business name, owner/contact details, phone/email, business type, address/location, and requested plan where applicable.
- New registration requests enter a pending state.
- Super Admin can review requests.
- Super Admin can approve, reject, suspend, or reactivate businesses.
- Approval should create/activate the required business membership/access without exposing platform secrets.
- Show clear status messages to applicants and admins.
- Record approval/rejection/status changes in the audit trail.

## 8. Super Admin panel

- Platform overview dashboard.
- Business list with search/filter/status.
- View business details.
- Approve/reject registration requests.
- Suspend/reactivate business accounts.
- View plan/status information.
- Platform-level activity/audit information.
- Basic metrics: total businesses, active businesses, pending requests, suspended businesses, and other useful platform metrics.
- Do not expose one business's private transactional data to another business.

## 9. Business dashboard

- Clean overview of today's sales.
- Sales count.
- Revenue.
- Items sold.
- Low-stock alerts.
- Out-of-stock alerts.
- Customer summary.
- Recent transactions.
- Quick actions for New Sale, Add Product, Add Customer, Stock Adjustment, and relevant workflows.
- Date-range controls where useful.
- Dashboard should be responsive and fast.

## 10. Product Management

- Product search bar.
- Product list/grid/table views as appropriate.
- Product image upload and preview.
- Product name.
- SKU/barcode.
- Category.
- Description.

- Cost price.
- Selling price.
- Current stock.
- Low-stock threshold.
- Unit of measure.
- Active/inactive status.
- Optional supplier association.
- Create, view, edit, archive/deactivate, and restore products where appropriate.
- Prevent accidental deletion when historical transactions depend on a product; prefer archive/deactivate.
- Validate prices, SKU uniqueness within the business where required, and stock-related fields.

## 11. Categories

- Create, edit, archive, and manage product categories.
- Business-level isolation.
- Search/filter where useful.
- Prevent destructive deletion when products depend on a category; offer safe reassignment/archive behavior.

## 12. POS Checkout — critical workflow

- Fast product search.
- Product grid with images.
- Category filtering.
- Add product to cart.
- Increase/decrease quantity.
- Remove item.
- Clear cart.
- Show stock availability and prevent overselling unless an explicit business setting allows it.
- Cart subtotal.
- Discount support with proper validation.
- Tax support should be configurable rather than assumed.
- Grand total.
- Payment method selection: cash and other configurable methods as appropriate.
- Amount received.
- Automatic change calculation.
- Complete sale.
- Generate receipt/print-friendly receipt.
- Sale confirmation.
- Update stock transactionally when sale is completed.
- Record payment and sale items.
- Prevent duplicate sale submission if the user double-clicks or loses/reconnects during submission.
- Handle failed transactions safely.
- Make the POS usable on desktop and tablet-sized screens.

### 13. Customers

- Customer list.
- Search.
- Add/edit customer.
- Phone/email/contact information.
- Purchase history.
- Customer total spending and useful summary metrics.
- Optional notes.
- Optional customer credit/receivable support if implemented.
- Customer data must remain business-isolated.

### 14. Suppliers and purchasing

- Supplier management.
- Supplier contact information.
- Purchase records.
- Purchase items.
- Cost price.
- Quantity received.
- Purchase totals.
- Stock increases when purchases are finalized.
- Purchase history and supplier history.
- Support purchase status such as draft/received/cancelled where appropriate.
- Maintain inventory movement records for stock received.

### 15. Inventory

- Current stock by product.
- Low-stock products.
- Out-of-stock products.
- Inventory valuation using a clearly defined method.
- Stock adjustments with reason.
- Stock movement history.
- Stock-in from purchases.
- Stock-out from completed sales.
- Returns/refunds affecting stock where appropriate.
- Manual adjustment permissions restricted to authorized roles.
- Never silently modify stock; important stock changes should have traceable movement records.

### 16. Reports

- Sales report.
- Daily/weekly/monthly/custom date-range sales.
- Top-selling products.
- Sales by product/category.

- Payment method summary.
- Customer sales summary.
- Inventory/stock report.
- Low-stock report.
- Purchasing/supplier report.
- Profit/margin reporting where cost data is available.
- Refund/return report.
- Dashboard KPIs.
- Use accurate financial calculations and date/time handling.
- Provide print-friendly report layouts and export options where practical.

## **17. Returns, refunds, credits and financial integrity**

- Design a safe return/refund workflow.
- Never silently edit historical sales to fake a refund.
- Record refunds/returns as traceable transactions.
- Update stock appropriately depending on whether returned goods are resellable.
- Record who performed the action and when.
- If customer credit/receivables are implemented, keep balances auditable and business-isolated.

## **18. Audit Trail**

- Record important actions such as login/security events where practical, product changes, price changes, stock adjustments, sales voids/cancellations, refunds, business status changes, role/permission changes, and settings changes.
- Store actor/user, business, action, entity/type, entity ID where appropriate, timestamp, and useful before/after metadata without exposing secrets.
- Audit records should be append-oriented and protected from ordinary users.
- Super Admin should have platform-level audit visibility; business admins should see their own business activity subject to permissions.

## **19. Offline resilience**

- Design the POS for unreliable internet conditions common in real-world retail environments.
- Provide clear online/offline indicators.
- Cache safe read-only data needed for POS operation where practical.
- Queue eligible transactions safely when offline.
- Synchronize when connectivity returns.
- Prevent duplicate sales during retries.
- Clearly communicate synchronization state.
- Do not claim full offline capability unless the implementation is actually safe and tested.
- Protect against conflicting inventory updates and failed synchronization.

## **20. Settings**

- Business profile/settings.

- Business name/logo/contact information.
- Currency configuration locked to GMD initially but architected for future expansion.
- Receipt settings.
- Tax settings if enabled.
- Low-stock defaults.
- User/role management.
- Security/session-related settings where appropriate.
- Notification preferences where appropriate.
- Branch/location settings where implemented.

## 21. UI/UX requirements

- Use Poppins throughout the product.
- Use Deep Green #0B3D2E as the primary brand color, Gold #D4AF37 as an accent, and White as the main light surface.
- Use accessible contrast and clear typography.
- Consistent sidebar/header/navigation.
- Responsive design.
- Professional cards, tables, forms, modals, dropdowns, badges, alerts, and empty states.
- Clear confirmation dialogs for destructive or financial actions.
- Toast/feedback messages for success and failure.
- Loading skeletons/spinners where appropriate.
- Avoid clutter and unnecessary animations.
- Prioritize speed and clarity over decorative design.

## 22. Security requirements

- Never place Supabase service-role keys in browser/client code.
- Never hard-code owner/admin passwords.
- Use RLS on all tenant-sensitive data.
- Validate authorization on the server/database side, not only in React.
- Validate user input.
- Protect sensitive operations by role/permission.
- Avoid exposing sensitive information in logs.
- Use secure database functions/RPC only when necessary and apply appropriate authorization checks.
- Review common risks: cross-tenant access, IDOR-style access, privilege escalation, unsafe file access, duplicate transaction submission, and insecure client-side assumptions.

## 23. Error handling and reliability

- No uncaught React errors on normal workflows.
- Friendly error messages.
- Useful developer diagnostics without exposing secrets.
- Handle empty data gracefully.
- Handle slow network requests.

- Disable duplicate submissions while requests are processing.
- Use transactional database operations for multi-step financial/stock changes where appropriate.
- Do not silently swallow errors.

## 24. Testing and quality assurance

- Run type checking.
- Run linting.
- Run production build.
- Test authentication and authorization.
- Test RLS/tenant isolation.
- Test CRUD workflows.
- Test POS sale completion.
- Test stock changes after sales and purchases.
- Test refunds/returns.
- Test reports.
- Test responsive layouts.
- Test offline/reconnect behavior where implemented.
- Fix errors rather than merely reporting them.
- Do not mark a phase complete until its acceptance criteria work.

## 25. Deployment philosophy

- Keep the application deployable using free/low-cost infrastructure during the early stage.
- Use Supabase Free during development/early launch where its current limits are sufficient.
- Do not require a custom domain for the initial launch.
- The initial product can be shared through its hosted application link.
- Do not add paid services unless necessary and explicitly approved.
- Keep environment configuration documented so hosting can be changed later without rewriting the application.

## 26. Phased implementation plan

1. Phase 1 — Foundation: initialize project, TypeScript, Vite, Tailwind, routing, design system, environment configuration, base layout, error/loading components.
2. Phase 2 — Supabase: schema, migrations, indexes, seed data, RLS, storage, database helper layer.
3. Phase 3 — Authentication: login, logout, sessions, protected routes, profiles, roles.
4. Phase 4 — Business onboarding: registration request, approval workflow, business membership, Super Admin foundation.
5. Phase 5 — Super Admin dashboard: businesses, statuses, approvals, platform overview, audit visibility.
6. Phase 6 — Business dashboard: KPIs, alerts, recent activity, quick actions.
7. Phase 7 — Products/categories: complete product and category management with images.
8. Phase 8 — POS: complete checkout workflow, payments, receipts, transactional stock updates.
9. Phase 9 — Customers: customer management and purchase history.
10. Phase 10 — Suppliers/purchases: suppliers, purchasing, receiving, stock-in.
11. Phase 11 — Inventory: stock dashboard, adjustments, movement history, low-stock controls.

12. Phase 12 — Returns/refunds/credit: safe financial and stock adjustments.
13. Phase 13 — Reports: sales, inventory, purchases, customers, margins and print/export.
14. Phase 14 — Audit Trail: complete important-event tracking and views.
15. Phase 15 — Offline resilience: online/offline state, safe caching/queue/sync where practical.
16. Phase 16 — Settings: business profile, receipt, tax, users/roles, branches and preferences.
17. Phase 17 — Security hardening: RLS review, permission review, secret review, cross-tenant penetration-style checks.
18. Phase 18 — QA and polish: testing, responsive fixes, accessibility, performance, empty/error/loading states.
19. Phase 19 — Deployment readiness: environment checklist, migrations, seed strategy, production build, deployment documentation.

## 27. Rules for every Claude implementation session

20. First inspect the repository and determine what is already implemented.
21. Read PROJECT\_SPEC.md / this specification and BUILD\_STATUS.md/TODO.md if present.
22. State briefly which phase you are implementing.
23. Implement only the requested phase unless a dependency must be fixed to make it work.
24. Do not delete or rewrite unrelated working functionality.
25. After implementation, run the relevant tests/typecheck/lint/build.
26. Fix errors found during verification.
27. Update BUILD\_STATUS.md/TODO.md with completed work, remaining work, known issues, and the exact recommended next phase.
28. Summarize changed files and important technical decisions.
29. Leave the project runnable.

## 28. First instruction to Claude

Start by inspecting the current workspace. If this is a new empty project, initialize JabangStore using the architecture and branding in this specification. Do not attempt to build all modules in one response. Begin with Phase 1 only. Create the permanent project documentation files (PROJECT\_SPEC.md and BUILD\_STATUS.md) and copy the essential requirements from this document into PROJECT\_SPEC.md so future Claude sessions can continue from the project itself. Then implement and verify Phase 1.

## 29. Acceptance standard — 10/10

- The application must look like a real commercial SaaS product, not a school/demo project.
- Core financial and inventory operations must be reliable and traceable.
- Multi-business isolation must be enforced by database security.
- The POS must be fast and simple for everyday shop use.
- The application must be usable by non-technical business owners.
- No major feature should depend on fake local arrays when it is supposed to use Supabase.
- No critical security secret may be exposed in client code.
- No phase is considered complete merely because a screen renders; the underlying workflow must function.
- The codebase must remain maintainable and ready for incremental expansion.
- JabangStore is the final product name throughout the user-facing experience.

### **30. Important note about the old JabangHub name**

The product is now JabangStore. Treat JabangStore as the canonical product name. Where legacy JabangHub references exist in an existing codebase, documentation, metadata, or UI, replace them safely. Do not accidentally change third-party package names, Supabase URLs, or unrelated technical identifiers merely because they contain a similar string.

## QUICK COPY-PASTE PHASE PROMPTS

Use these short prompts after placing this master specification in the project. They are intentionally small so Claude does not need the entire specification pasted into every chat.

### Phase 1

Read PROJECT\_SPEC.md. Implement Phase 1 only: project foundation, React + TypeScript + Vite, Tailwind, routing, base layout, design system, environment configuration, loading/error/empty states. Verify with typecheck/lint/build and update BUILD\_STATUS.md.

### Phase 2

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 2 only: Supabase PostgreSQL schema, migrations, indexes, seed data, RLS, storage, and database helper layer. Test tenant isolation carefully. Do not build later UI modules.

### Phase 3

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 3 only: Supabase authentication, sessions, protected routes, profiles, roles and permissions. Do not expose secrets.

### Phase 4

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 4 only: business registration request, pending/approved/rejected workflow, memberships, and Super Admin foundation.

### Phase 5

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 5 only: Super Admin dashboard for businesses, approvals, statuses, platform metrics and audit visibility.

### Phase 6

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 6 only: business dashboard with sales KPIs, low-stock alerts, recent transactions and quick actions.

### Phase 7

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 7 only: product/category management, images, search, validation, archive/deactivate behavior and business isolation.

### Phase 8

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 8 only: complete POS checkout, cart, quantities, stock validation, discounts/tax configuration, payments, change, sale completion, receipt and transactional stock updates.

### Phase 9

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 9 only: customer management, search, profiles, purchase history and business isolation.

### Phase 10

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 10 only: supplier management, purchasing, purchase items, receiving and stock-in.

#### **Phase 11**

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 11 only: inventory dashboard, stock adjustments, movement history, low-stock and out-of-stock workflows.

#### **Phase 12**

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 12 only: returns, refunds and any approved customer-credit workflow with full auditability.

#### **Phase 13**

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 13 only: sales, inventory, purchasing, customer, margin and print/export reports.

#### **Phase 14**

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 14 only: complete audit trail for important actions and appropriate admin views.

#### **Phase 15**

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 15 only: offline resilience, safe caching/queue/sync where practical, online/offline indicator and duplicate-prevention.

#### **Phase 16**

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Implement Phase 16 only: business settings, receipt settings, tax settings, users/roles, branches and preferences.

#### **Phase 17**

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Perform Phase 17 security hardening: review RLS, authorization, tenant isolation, secrets, file access, privilege escalation and duplicate financial transactions. Fix issues found.

#### **Phase 18**

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Perform Phase 18 QA/polish: typecheck, lint, build, workflow testing, responsive fixes, accessibility, performance and error/loading/empty states.

#### **Phase 19**

Read PROJECT\_SPEC.md and BUILD\_STATUS.md. Prepare Phase 19 deployment readiness: environment checklist, database migration process, production build, deployment documentation and safe configuration.

## Final instruction

Do not optimize for producing the most code in one response. Optimize for producing correct, secure, tested, maintainable software that can be continued by another Claude session. If the available context becomes limited, save the current state in BUILD\_STATUS.md before stopping.